

Architecture Documentation — Sensing Garden Dashboard

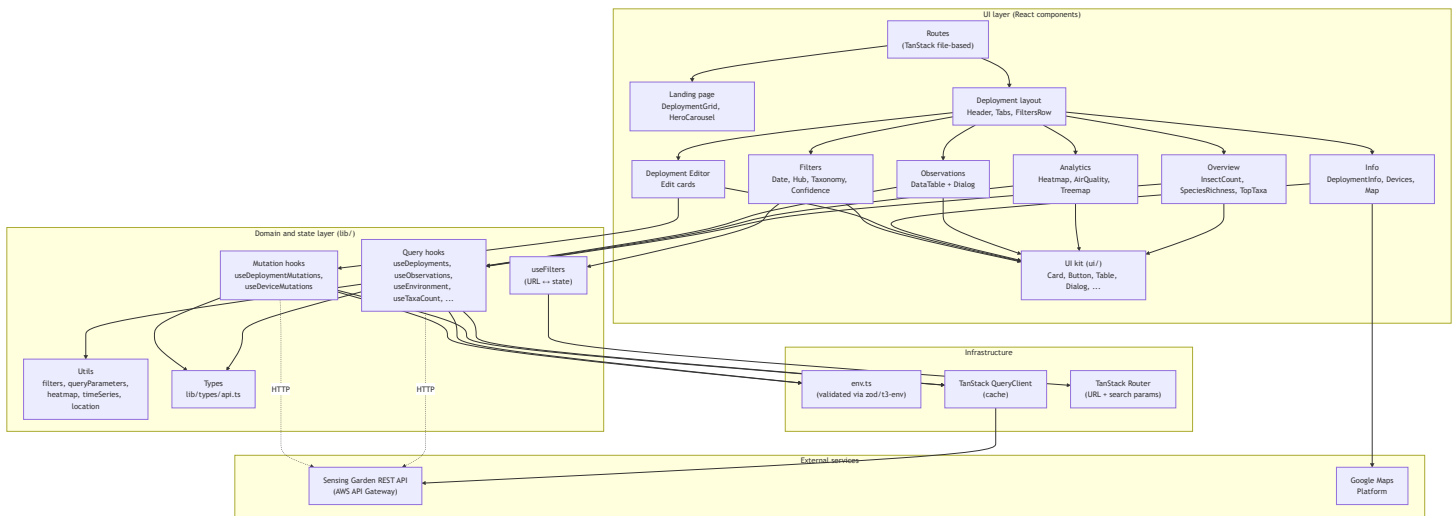
The architecture is described using Kruchten's **4+1 View Model**. Each view answers a different question:

View	Question	Audience
Logical	What does the system consist of, functionally?	End users, designers
Process	How does the system behave at runtime?	Integrators
Development	How is the code organized?	Developers
Physical (Deployment)	Where does it run?	Operations / sysadmin
Scenarios (+1)	How does it work in practice?	Everyone

The dashboard is a React 19 SPA (Vite + TanStack Router/Query) that visualizes data from the Sensing Garden backend. All state lives either in the URL (filters via query params) or in the TanStack Query cache (server data). There is no separate backend in this repo — the app talks directly to the REST API on AWS API Gateway.

1. Logical View

Shows the main components, their responsibilities, and dependency directions. The layering mirrors the folder structure under `src/`.

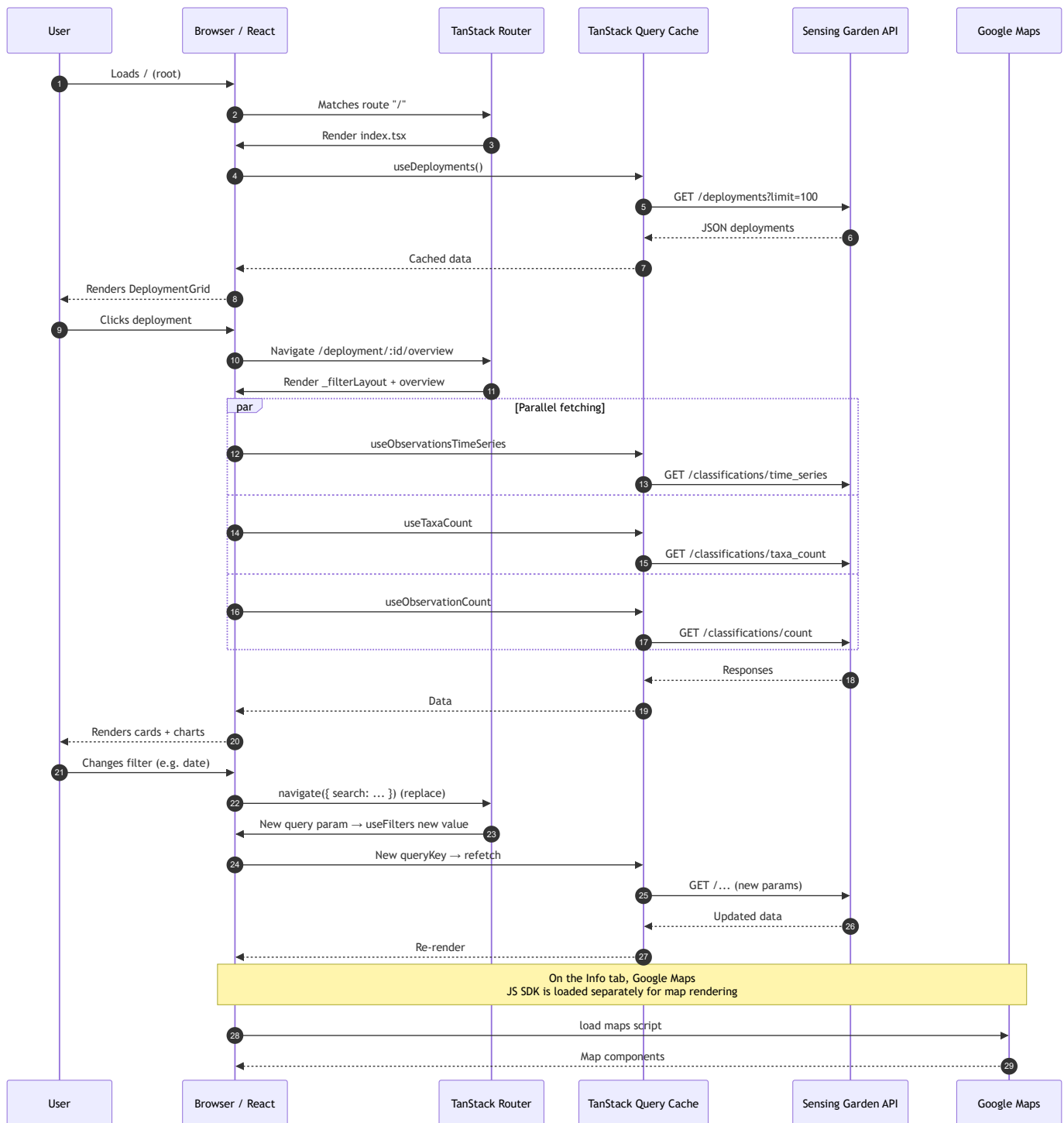


Key abstractions

- **Routes** — one file per URL under `src/routes/`. `_filterLayout` is a "pathless layout route" that shares Header/Tabs/FiltersRow across all sub-pages for a single deployment.
- **Query hooks** — thin wrappers around `fetch` + `useQuery`. One hook per endpoint. Response types are centrally defined in `lib/types/api.ts`.
- **useFilters** — reads/writes filter state to URL search parameters, validated with Zod (`filtersSchema`). Enables shareable links and browser "back" support.
- **UI kit** (`components/ui/`) — shadcn-inspired Base UI components. Reusable building blocks with no domain knowledge.

2. Process View

Shows runtime behavior: startup, data flow, and concurrent fetches. The app has only one process (a browser tab), but multiple async operations are coordinated by TanStack Query.



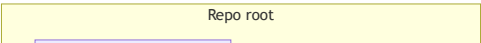
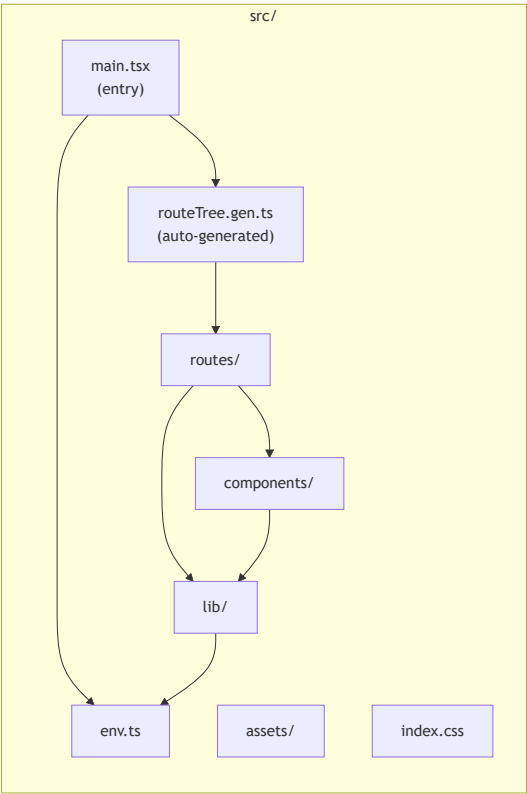
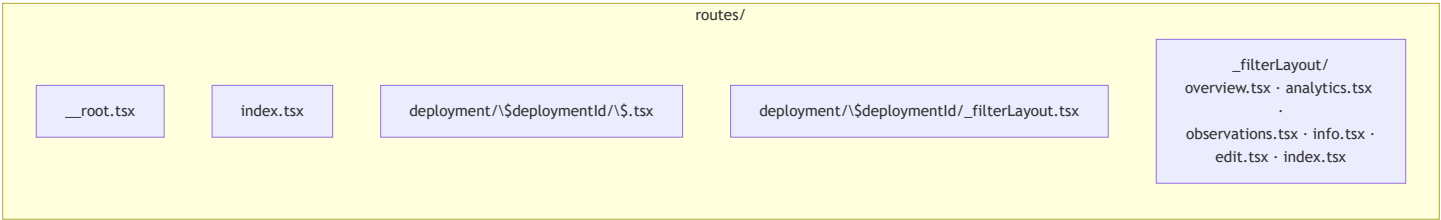
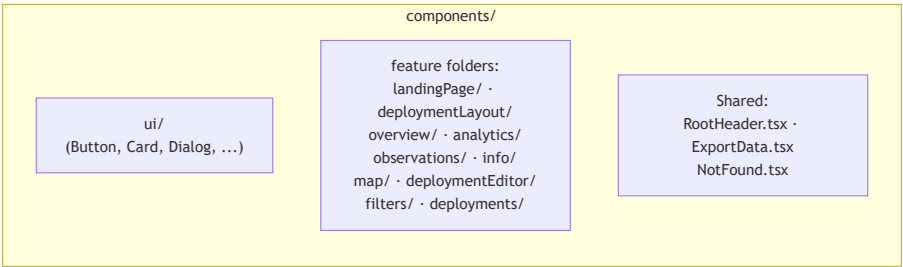
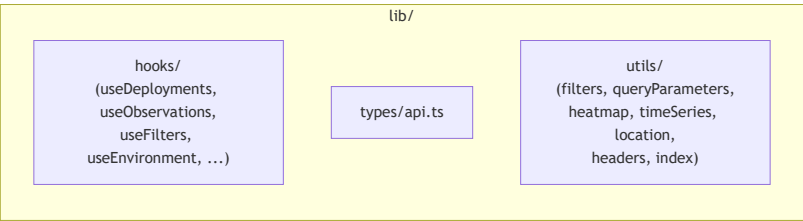
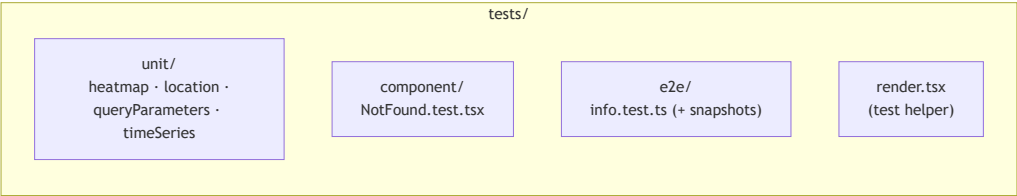
Characteristics

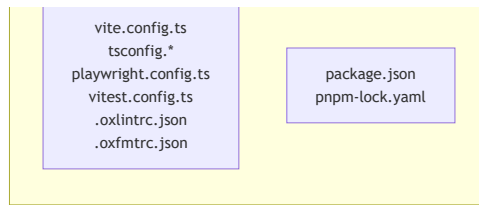
- **No global state manager** — the cache is the source of truth for server data; the URL is the source of truth for filter state.
- **Cache invalidation** happens after mutations (`queryClient.invalidateQueries(["deployments"])` , etc.).
- **Preload on intent** — router has `defaultPreload: "intent"` , so hovering a link pre-fetches route data.

- **Parallel fetch** — independent hooks in the same render trigger parallel requests; React Query dedupes them.
- **Export** (`ExportData`) performs an ad-hoc fetch for ZIP/CSV/JSON and uses `JSZip + Blob +` object URLs for client-side download.

3. Development View

Source code organization as seen by the developer. The alias `@/` points to `src/`.





Conventions (from CONTRIBUTING.md)

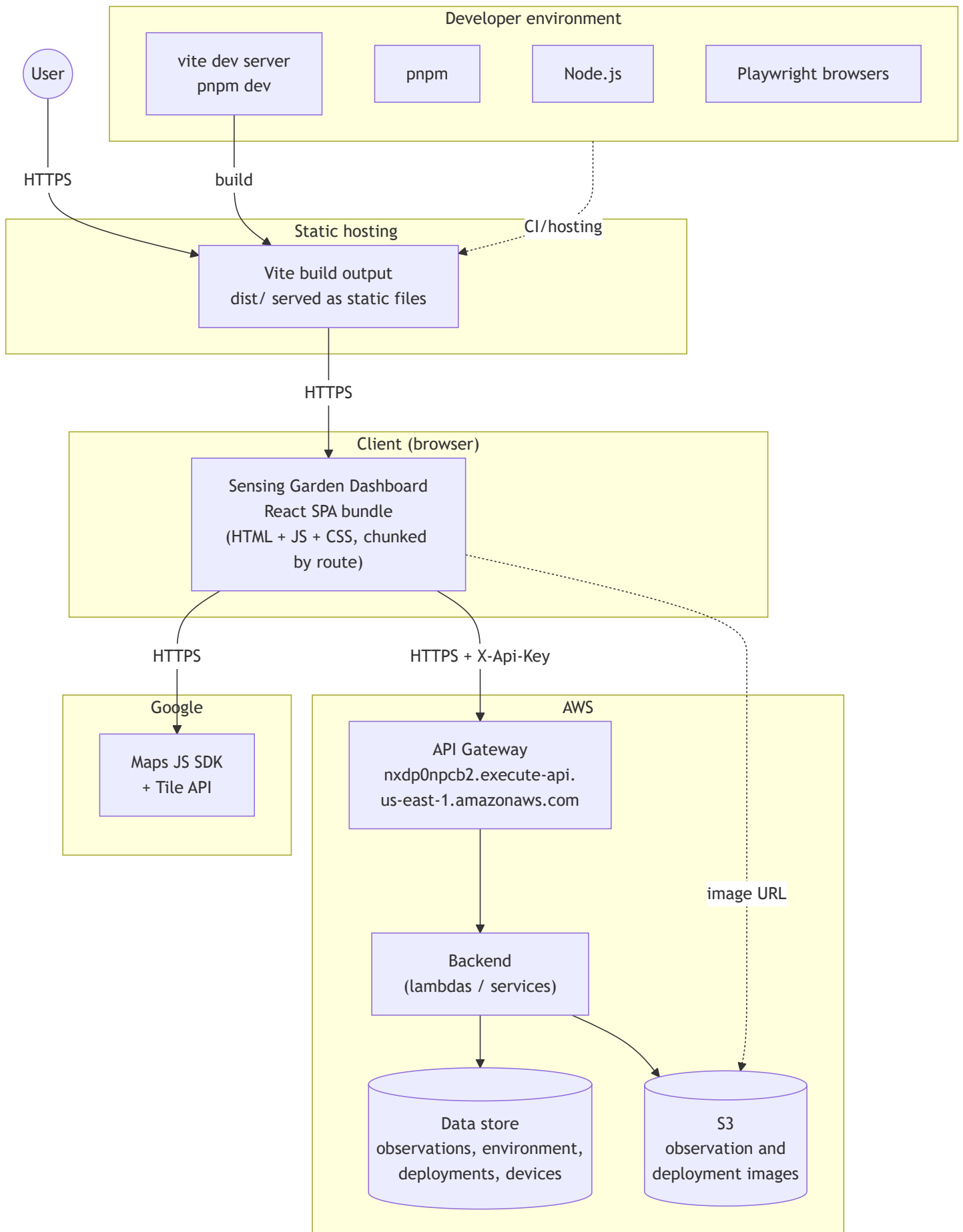
- React components use PascalCase and live in `src/components/<feature>/` .
- UI primitives from shadcn (Base UI variant) go in `components/ui/` .
- Data fetching uses TanStack Query hooks in `lib/hooks/` .
- No inline `export` — a single `export { ... }` at the bottom of the file (except `Route` in routes).
- Tailwind for styling, with custom theme tokens (`text-muted-foreground` , not `text-zinc-400`).
- Oxlint + Oxfmt run via `simple-git-hooks` as a pre-commit hook.

Test pyramid

- Unit (`vitest`): pure utility functions.
- Component (`vitest` + `vitest-browser-react`): top-level components.
- E2E (`playwright`): route/page level, including visual snapshots.

4. Physical View (Deployment)

Where does the application run in production?



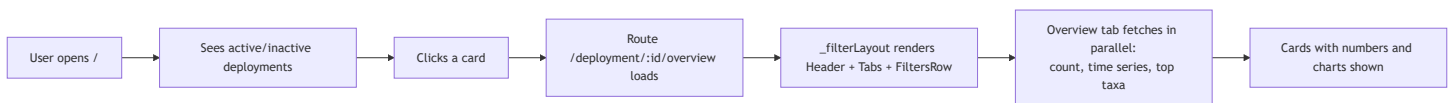
Runtime assumptions

- The SPA bundle only needs static hosting (CDN / object storage) — no server-side rendering.
- `.env` must contain `VITE_API_BASE_URL`, `VITE_API_KEY`, and optionally `VITE_GOOGLE_MAPS_API_KEY`. These are baked into the bundle by `vite build` and are therefore public. The API key should consequently be restricted on the server side.
- Authentication: every call attaches the `X-Api-Key` header (see `lib/utis/headers.ts`).

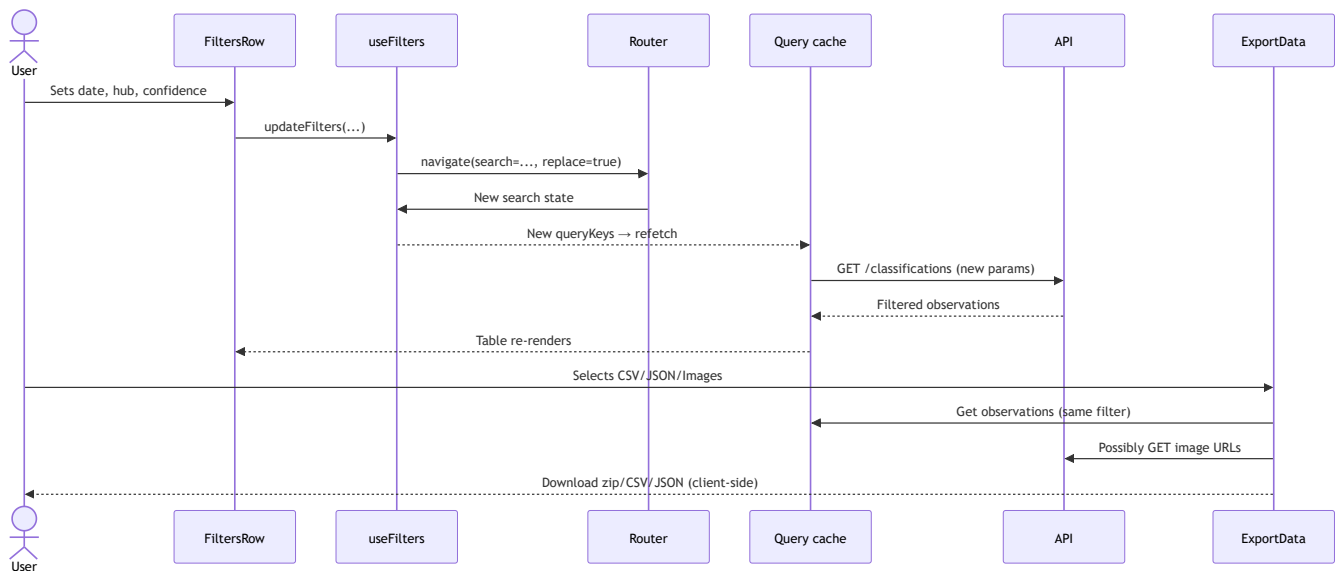
5. Scenarios (+1)

Three core scenarios that tie the other views together.

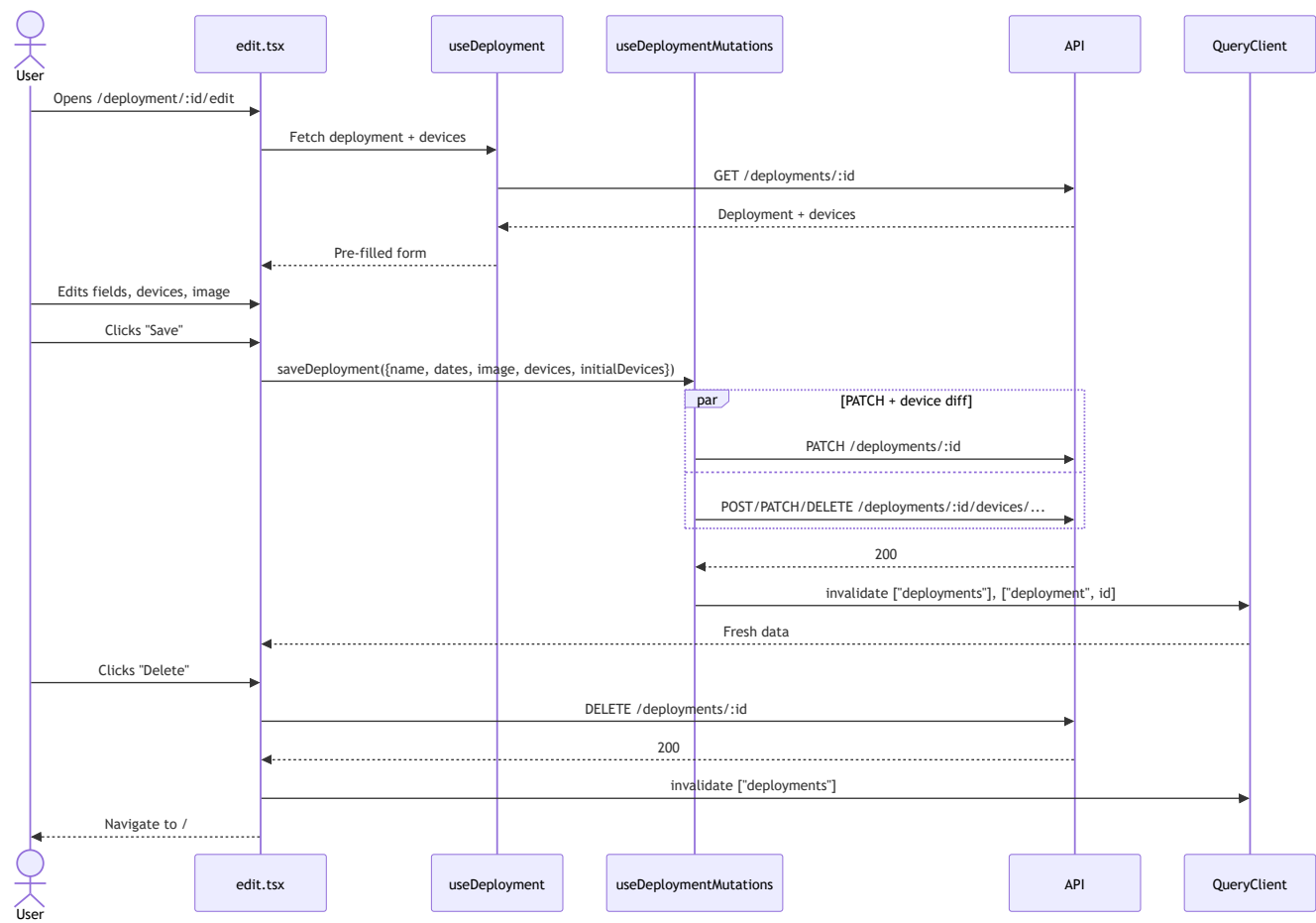
Scenario A — View data for a deployment



Scenario B — Filter observations and export



Scenario C — Edit deployment (CRUD)



Appendix — Key technologies

Area	Choice
Language/runtime	TypeScript, React 19, Node (dev), pnpm
Bundler/dev	Vite 8, @vitejs/plugin-react , @tailwindcss/vite
Routing	@tanstack/react-router (file-based) + @tanstack/zod-adapter
Data/cache	@tanstack/react-query
Table	@tanstack/react-table
UI	Base UI (@base-ui/react), Tailwind v4, class-variance-authority , lucide-react
Maps	@vis.gl/react-google-maps

Area	Choice
Charts	recharts
Validation	zod , @t3-oss/env-core
Testing	Vitest, vitest-browser-react , Playwright
Lint/fmt	Oxlint, Oxfmt, simple-git-hooks , lint-staged